
Twin Synergy Telecommunications

Tor + Blockchain Architecture

Sol in Libra, 2026 e.v.
Johnathan 'Qasparr' (Κασπάρι) Monroe,
Keeper of the Secret Treasure

93

Twin Synergy Telecommunications — Founding Design

"Unfortunately, no one can be told what the Matrix is. You have to see it for yourself."

— Morpheus, *The Matrix* (The Wachowskis, 1999) [VERIFIED FACT]

Date: Friday (Dies Veneris), Sol in Libra, Anno 2026 e.v. — 9 October 2026 **Authorship:** Johnathan "Qasparr" (Κασπάρρ) Monroe, Keeper of the Secret Treasure **Method:** Scientific Illuminism. **Status:** [DESIGN] — founding design stage; Twin Synergy is a design, not a legal filing. No chain is decreed herein; chain choice is his red pen's alone.

93.

Status-mark legend (used throughout)

- ****[VERIFIED FACT]**** — checked against a live source, the text on disk, or his recorded ruling.
 - ****[DESIGN]**** — the scribe's proposed architecture; stands until his red pen rules otherwise.
 - ****[HYPOTHESIS]**** — a reasoned but unverified claim; flagged, never smuggled as fact.
 - ****[SCRIBE]**** — the scribe's own working note, uncertainty, or adaptation point.
-

§I. Doctrines inherited (not re-argued)

These arrive from his prior verified work and bind this design:

- ****TOFU identity doctrine.**** Trust-on-first-use was kept; out-of-band identity pinning was declined [VERIFIED FACT — his ruling]. First contact records the key; all later contact must prove it.
 - ****Real cryptography only.**** His PQPE red-pen ruling: hybrid ****X25519+ML-KEM-768**** / ****Ed25519+ML-DSA-65**** where possible; otherwise real classical-only with explicit opt-in. No fake, demo, or theater crypto ships [VERIFIED FACT — his ruling].
 - ****Loopback-only enclave + Tor v3 onion front.**** Local services bind `127.0.0.1` only; the world meets them through v3 onion services [VERIFIED FACT — his oz doctrine].
 - ****Fail-closed.**** Archive/verification failure is loud, never silent [VERIFIED FACT — his oz doctrine].
 - ****Hash-chained JSONL ledger as integrity anchor**** [VERIFIED FACT — his oz design].
 - ****The public CA model is malicious by default**** [VERIFIED FACT — his stated position, 2026-10-01]. This design therefore never makes a CA signature the root of trust. DANE/DNSSEC-first; user-held keys; transparency for anything the registrar itself signs.
-

§II. The Tor layer — onion-first, clearnet optional

§II-A. Portals as v3 onion services [DESIGN]

Every customer portal — account, billing, support — is a **Tor v3 onion service**. There is no clearnet requirement to be a Twin Synergy customer. The backends bind 127.0.0.1 only (loopback doctrine, §I.3); Tor forwards into the enclave.

v3 onion addresses are 56-character base32 and **self-authenticating**: the address is the ed25519 public key (plus checksum and version byte) [VERIFIED FACT]. A customer who knows the address cannot be redirected to an impostor — the name is the key.

Each portal gets its **own onion service, its own keypair, its own Tor instance** — blast-radius isolation: a compromise of the support portal's key does not touch billing [DESIGN].

Thelemic-annotated torrc — portal onion service [DESIGN]

```
# =====
# TORRC – Twin Synergy portal onion service (illustration)
# Johnathan "Qasparr" (Κασπάρρ) Monroe, Keeper of the Secret Treasure
# Friday, Sol in Libra, 2026 e.v. – 93.
# Method: Scientific Illuminism. What can occur – will.
# =====

# MECHANISM: Run as a client only. This machine is an onion-service
# host, not a relay; it contributes no transit bandwidth from here.
# DOCTRINE: Separation of powers – relay duty lives on relay hosts
# ($V), service duty lives here. One host, one office.

ClientOnly 1

# MECHANISM: SOCKS port bound to loopback only, for local tooling
# (curl over SOCKS, health checks). Never 0.0.0.0.
# DOCTRINE: Loopback-only, his standing doctrine (§I.3).
SOCKSPort 127.0.0.1:9050

# MECHANISM: HiddenServiceDir holds the service's ed25519 keypair
# and hostname file. Tor generates them on first start if absent.
# DOCTRINE: The keypair is generated once and backed up like a
# crown jewel – the .onion address derives from it; lose the key,
# lose the name forever. No CA can reissue it. That is the point.
HiddenServiceDir /var/lib/tor/tsyn_portal_account/
HiddenServiceVersion 3

# MECHANISM: Forward onion-port 80 to the loopback-only backend.
# The backend (account server) listens on 127.0.0.1:8080 and
# refuses every non-loopback connection.
# DOCTRINE: The enclave never faces the network; Tor is the only
# door, and Tor checks papers at the rendezvous (see §II-C).
HiddenServicePort 80 127.0.0.1:8080

# MECHANISM: A second service for billing, separate keypair.
# DOCTRINE: Blast-radius isolation (§II-A). Compromise of one
# portal's key buys the attacker nothing about the others.
HiddenServiceDir /var/lib/tor/tsyn_portal_billing/
HiddenServiceVersion 3
HiddenServicePort 80 127.0.0.1:8081
```

```
# MECHANISM: Support portal, same pattern, third keypair.
HiddenServiceDir /var/lib/tor/tsyn_portal_support/
HiddenServiceVersion 3
HiddenServicePort 80 127.0.0.1:8082
```

§II-B. Onion-first, clearnet optional [DESIGN]

- The `.onion` address is **canonical**. It is printed on invoices, in the welcome letter, and pinned in the customer's DID document (§III).
- Clearnet, if offered at all, is a convenience mirror, never the authority: it serves an `Onion-Location` header pointing at the canonical onion [VERIFIED FACT — this header mechanism exists and is standard practice], and it carries no function the onion lacks.
- **Fail-closed rule:** if the onion service fails to bootstrap, the portal is **down**. It does not "fail over" to clearnet-only operation, because silent downgrade from an authenticated name to DNS-and-CA trust would be a fail-open [DESIGN].

§II-C. Guard practices [DESIGN, grounded in VERIFIED FACT]

- **vanguards** (the Tor Project's onion-service guard-protection addon) on every service host — it defends against guard-discovery attacks [VERIFIED FACT — vanguards exists for this purpose].
- Never run a public relay and an onion service on the same host: relay operators are public by design; service hosts must not be correlatable [DESIGN].
- Guard rotation left to Tor's defaults; Tor kept current; **no access logs** on the onion front — logs are a liability inventory (cf. §III data minimization).
- Per-portal Tor instances (above) so circuit state never mingles across portals.

§II-D. Onion-service authentication for SSH [DESIGN]

Administrative SSH rides onion too — no clearnet SSH, no open port 22 anywhere. Two gates, both real cryptography:

- **Tor-layer gate** (v3 client authorization). The service's `authorized_clients/` directory holds one `.auth` file per administrator, each a single line [VERIFIED FACT — torrc.5]:

```
descriptor:x25519:<BASE32-ENCODED-ADMIN-PUBLIC-KEY>
```

Once any valid `.auth` file loads, the service answers only to holders of matching private keys; everyone else fails at the rendezvous with no distinguishable error [VERIFIED FACT]. The x25519 keypair is generated with a real X25519 implementation — never hand-rolled, never demo (§I.2) [SCRIBE: operator tooling choice; any audited X25519 implementation qualifies].

- **SSH-layer gate.** The administrator's client carries the private key in `ClientOnionAuthDir`:

```
# ~/.tor/onion_auth/<onion-address-without-.onion>.auth_private
<onion-address>:descriptor:x25519:<BASE32-ENCODED-PRIVATE-KEY>
```

and connects through Tor's SOCKS proxy (illustration — adapt `ncat/nc` to the operator's platform) [SCRIBE]:

```
# ~/.ssh/config – Twin Synergy admin SSH over onion
```

```

Host tsyn-admin
  HostName <56-char-onion-address>.onion
  # MECHANISM: Push the SSH handshake through Tor's loopback SOCKS.
  # DOCTRINE: SSH never touches clearnet; the Tor layer has already
  # demanded the x25519 client credential before SSH speaks at all.
  ProxyCommand ncat --proxy 127.0.0.1:9050 --proxy-type socks5 %h %p
  # MECHANISM: ed25519 host key, pinned TOFU on first contact.
  # DOCTRINE: His TOFU ruling (§I.1) — first sight records the key;
  # every later sighting must match it or the connection dies loudly.
  User admin

```

Inside, authentication is ed25519 SSH keys (hybrid PQ per §I.2 where the PQPE ruling applies — root infrastructure keys). Password authentication is disabled outright [DESIGN].

§III. Blockchain / identity layer — DID-style, TOFU-rooted

§III-A. Identity = TOFU + cryptographic proof [DESIGN]

There is **no central identity database** — that is the data-minimization doctrine, and it is load-bearing:

- At signup, the customer generates an **ed25519 identity keypair locally** (in their own client, on their own device — the registrar never sees the private key) and presents the public key over the onion portal.
- The registrar records the public key on **first contact (TOFU)** and binds it to the account [DESIGN, per his TOFU ruling §I.1].
- Every later authentication is **proof-of-possession**: the portal issues a challenge; the customer signs it. No passwords stored, no password database to breach — there is nothing to steal [DESIGN].
- The customer's public identity record is a **DID-style document** (W3C DID Core data-model shape [VERIFIED FACT — the standard exists]; the method name is **TBD** — his red pen, so the illustration below uses the W3C's own `did:example` placeholder honestly):`

```

{
  "@context": "https://www.w3.org/ns/did/v1",
  "id": "did:example:123456789abcdefghi",
  "verificationMethod": [{
    "id": "did:example:123456789abcdefghi#key-1",
    "type": "Ed25519VerificationKey2020",
    "controller": "did:example:123456789abcdefghi",
    "publicKeyMultibase": "zH3C2AVvLMv6gmMnam3uVAjZpfkcJCwDwnZn6z3wXmqPV"
  }],
  "authentication": ["did:example:123456789abcdefghi#key-1"],
  "service": [{
    "id": "did:example:123456789abcdefghi#portal",
    "type": "OnionPortal",
    "serviceEndpoint": "http://<canonical-56-char>.onion"
  }]
}

```

MECHANISM (for the student): a DID document is a public directory card — "this identifier is controlled by whoever holds the private half of this public key, and here is where to reach them." DOCTRINE: the card is public by design; the key is private by design; the registrar is a printer of cards, not a keeper of keys.

§III-B. Key management — the honest parts [DESIGN]

- **Generation:** client-side, real randomness, ed25519 today; hybrid **Ed25519+ML-DSA-65** for registrar root and recovery keys per his PQPE ruling (keys that must survive decades face the quantum horizon) [DESIGN].
- **Rotation:** a signed rotation statement, chained into the ledger (§III-D) — the old key attests the new key, TOFU chain unbroken [DESIGN].
- **Storage:** the registrar stores public keys and DID documents only. Private keys live with the customer. Full stop.

§III-C. Recovery — the hard parts, stated plainly [DESIGN]

This is the hardest problem in the design, and the document will not pretend otherwise:

- **The sovereign truth:** if the customer loses their private key and there is no recovery path, the identity — and every domain bound to it — is **unrecoverable**. No "forgot password" email can fix mathematics. That is the price of "no central identity database," and the customer must be told it in plain words at signup [DESIGN].
- **Option 1 — Shamir-split recovery shares.** The recovery secret is split N-of-M (Shamir's Secret Sharing [VERIFIED FACT — the scheme exists]); one share escrowed with the registrar, others with the customer's chosen trustees. **Honest cost:** the escrowed share re-centralizes a fragment of power — say so on the tin [DESIGN].
- **Option 2 — Social recovery.** N-of-M trusted contacts co-sign a rotation to a new key. **Honest cost:** your trustees become your attack surface; choose them like your life depends on it, because your domains do [DESIGN].
- **Option 3 — No recovery (sovereignty maximalist).** Key loss = identity death. **Honest cost:** domains die with keys; estates, businesses, and forgetful humans all lose [DESIGN].
- **No decree here.** The recovery model is a red-pen question (§VIII.2). The scribe's counsel: offer Option 1 as default with loud disclosure, Option 3 as the explicit opt-in for those who want it.

§III-D. The hash-chained JSONL ledger — integrity anchor [DESIGN]

Every registrar action — registration, key rotation, transfer, TLSA publication, takedown — is appended to a **hash-chained JSONL ledger** (his oz design, §I.5): each entry commits to the previous entry's hash. Verification failure **halts issuance loudly** (fail-closed, §I.4). Below is a real, stdlib-only sketch of the mechanism — no fake crypto, hashlib.sha256 throughout:

```
# =====
# LEDGER — hash-chained JSONL append/verify (teaching sketch)
# Johnathan "Qasparr" (Κασπάρι) Monroe, Keeper of the Secret Treasure
# Friday, Sol in Libra, 2026 e.v. — 93.
# Method: Scientific Illuminism. Live, Love, and let Love, Live.
# =====
```

```

import hashlib, json, sys, time

# MECHANISM: One SHA-256 over the canonical JSON of the previous
# entry plus the new payload. Each link commits to the whole chain
# behind it – tamper with entry 5 and every link after it breaks.
# DOCTRINE: The registrar's power exercised in daylight. A silent
# edit is mathematically impossible; a loud halt is the design.
def _digest(prev_hash: str, payload: dict) -> str:
    # MECHANISM: sort_keys + compact separators = canonical bytes;
    # without canonicalization, identical entries hash differently.
    body = json.dumps(payload, sort_keys=True,
                       separators=(",", ":")).encode()
    return hashlib.sha256((prev_hash + body.decode()).encode()
                          ).hexdigest()

def append(path: str, action: str, data: dict) -> None:
    # MECHANISM: Read the tail hash; genesis uses 64 zeros.
    try:
        with open(path) as f:
            lines = f.read().strip().split("\n")
            prev = json.loads(lines[-1])["hash"] if lines[0] else "0" * 64
    except FileNotFoundError:
        lines, prev = [], "0" * 64
    entry = {"ts": int(time.time()), "action": action,
            "data": data, "prev": prev}
    entry["hash"] = _digest(prev, {k: v for k, v in entry.items()
                                   if k != "hash"})
    # MECHANISM: Append-only. The file is never rewritten in place.
    with open(path, "a") as f:
        f.write(json.dumps(entry, sort_keys=True) + "\n")

def verify(path: str) -> None:
    # MECHANISM: Recompute every link. Any mismatch -> loud halt.
    # DOCTRINE: Fail-closed (§I.4). A broken chain stops issuance;
    # it never "warns and continues," because a warning nobody
    # reads is a fail-open with better manners.
    prev = "0" * 64
    with open(path) as f:
        for n, line in enumerate(f, 1):
            e = json.loads(line)
            if e["prev"] != prev:
                sys.exit(f"LEDGER BREAK at entry {n}: prev-link mismatch")
            if e["hash"] != _digest(
                prev, {k: v for k, v in e.items() if k != "hash"}):
                sys.exit(f"LEDGER BREAK at entry {n}: hash mismatch")
            prev = e["hash"]
    print(f"LEDGER VERIFIED through entry {n}")

# NOTE [SCRIBE]: teaching sketch, stdlib-only, real SHA-256 throughout.
# Production adds file locking, signed checkpoints, and the
# chain-anchor of §III-E. The mechanism above is not the whole
# fortress; it is the honest foundation stone.

```

§III-E. Chain survey — anchoring options, NO decree [DESIGN]

The ledger's periodic checkpoint hash can be anchored to an external chain for timestamped immutability. The scribe surveys; **his red pen decrees** (§VIII.1):

Option	Mechanism	Strengths	Honest costs
**A. Bitcoin-anchored ** (Sidetree/ION pattern)	Batch checkpoint hashes into Bitcoin transactions	Strongest timestamp security on earth; no new chain to bootstrap	Fee volatility; block-space politics; throughput is checkpoint-rate only
**B. Ethereum L2 anchor **	Checkpoint hash to a cheap L2 contract	Cheap, programmable, wide tooling	Token/bridge risk; EVM complexity; L2 sequencer trust assumptions
**C. Own chain ** (Tendermint-style BFT)	Twin Synergy validators finalize checkpoints	Full control; no fee market; fits the ISP footprint	**Who watches the validators?* Bootstrap trust problem; a chain with three validators is a database with marketing
**D. No chain ** — ledger + DNSSEC/DANE only	Signed checkpoints published in DNS, pinned by DANE	Simplest; honest; his oz design already works this way	No external timestamp authority; disputes resolve to "whose ledger verifies"
**E. OpenTimestamps **	Bitcoin attestation without running anything	Bitcoin-grade timestamps, near-zero cost, no custody	Attestation only — proves <i>*when*</i> , not <i>*what was valid*</i>

[HYPOTHESIS]: for a registrar, **D (no chain) with E (OpenTimestamps) as a cheap upgrade path** is the honest minimum — the ledger plus DNSSEC already carries the trust, and an external timestamp only hardens disputes. But the trade is his to call, not the scribe's.

§IV. Trust model — the registrar as trustee, never overlord

§IV-A. The legal shape of the office

"TRUST. 1. An equitable or beneficial right or title to land or other property, held for the beneficiary by another person, in whom resides the legal title or ownership, recognized and enforced by courts of chancery."

— Black's Law Dictionary, 2d ed. (1910), s.v. "TRUST" [VERIFIED FACT — public domain; verified in ~/workspace/blacks-2e/source/blacks-2e-1910_djvu.txt]

Twin Synergy holds the **bare legal title** of the zone entry — the registration record. The **equitable ownership** — the thing of value — is control of the registrant's key, which the registrar never holds [DESIGN]. This is the anti-overlord mechanism stated as architecture:

- **Registration**** = the registrant proves possession of their key (signed challenge); the registrar records the binding **key ↔ domain** in the ledger. The registrar cannot register a domain **to itself** without the ledger showing it — daylight [DESIGN].
- **Transfer**** = a key ceremony: the current key-holder signs a transfer statement naming the new key; the registrar countersigns and chains it. The registrar ****cannot transfer what it does not hold the key for**** — this is a designed incapacity, not a policy promise [DESIGN].
- **Takedown / suspension**** (court order, abuse) = a ledger-visible event, signed by the registrar, citing the order. Power exercised in daylight, or not at all. Unilateral silent seizure

is architecturally impossible without breaking the chain — and a broken chain halts the whole machine (§III-D) [DESIGN].

§IV-B. DANE / DNSSEC-first, CA-never-required

- Every Twin Synergy zone is ****DNSSEC-signed****; the KSK ceremonies are public and logged [DESIGN].
- Registrants publish ****TLSA records**** pinning *their own* keys (DANE) [VERIFIED FACT — the DANE/TLSA mechanism is the standard for this]. A customer needs ****no CA-issued certificate**** to be a Twin Synergy customer in good standing — the CA is not in the trust path [DESIGN, from his §I.6 position].
- Where the registrar issues any certificate at all (e.g., for its own portals), it is logged to ****Certificate Transparency**** — sunlight as disinfectant [DESIGN].
- The registrar ****cannot forge a TLSA**** for a registrant's domain without the registrant's key — the signature on the record set requires the zone-signing key for the *delegation*, but the TLSA *content* commits to the registrant's key, and any substitution is ledger-visible (§IV-A) [DESIGN].

§IV-C. How domain ownership is proved

- ****Claim:**** registrant signs a challenge with the identity key (TOFU, §III-A).
 - ****Record:**** the binding enters the hash-chained ledger.
 - ****Anchor:**** the ledger checkpoint is published (and optionally chain-anchored per §III-E).
 - ****Dispute:**** the ledger verifies, or it does not. There is no "registrar's word" above the mathematics — TRVVTH alone [DESIGN].
-

§V. Integration — how Tor + blockchain ride the network

§V-A. Onion-aware PoPs [DESIGN]

- PoPs host the vanguards-protected onion backends (§II) and the ledger/anchor infrastructure (§III).
- PoPs ****may**** run Tor ****middle/guard relays**** to contribute bandwidth to the network they depend on — honest reciprocity. Relay hosts are separate machines from onion-service hosts (§II-C) [DESIGN].
- Onion services need no special ISP magic: they ride the public Tor network. The design says this plainly rather than inventing "onion-aware routing" theater [SCRIBE].

§V-B. No exit nodes — the liability trap, stated honestly [DESIGN]

- Twin Synergy operates ****no exit relays****. Exit traffic emerges onto clearnet *from the exit's IP address* — abuse complaints, DMCA notices, and law-enforcement inquiries attach to that address [VERIFIED FACT — this is the well-documented operational reality of exit relaying].
- The design refuses the trap by architecture, not by promise: no exit relay configured, no exit policy to misconfigure.

- Middle/guard relays are lower-risk but ****not no-risk****; relay operation in any jurisdiction deserves counsel's review. ****Counsel required**** — this document is a design, not legal advice [SCRIBE].

§V-C. Legal notes (counsel required; not legal advice)

- Operating onion services is generally lawful, but obligations vary by jurisdiction (data-retention rules, lawful-intercept regimes, registrar duties). ****Counsel required**** before first customer.
- Registrar legal duties (abuse handling, court orders) are real; §IV-A's daylight rule makes compliance **visible**, which is a feature for counsel, not a bug [DESIGN].
- ****Root posture is undecided:**** ICANN-root compatibility vs. an independent/alternate root is a sovereignty question with legal and interoperability consequences. Red-pen question §VIII.4. ****Counsel required**** either way [SCRIBE].

§VI. Threat model + fail-closed behaviors

Threats (honest enumeration)

Threat	Mitigation in this design	Residual
Guard discovery / traffic correlation by a global passive adversary	vanguards; per-portal Tor instances; no relay+service colocation	A GPA is never fully mitigated — stated, not hidden [HYPOTHESIS: acceptable for portal traffic; not for life-safety anonymity]
Phishing of onion addresses	Addresses are self-authenticating; distributed via DID docs + invoices + DANE	First-sight TOFU still requires a trustworthy first channel — his accepted trade (§I.1)
Registrar key compromise (zone-signing / ledger-signing)	PQ-hybrid root keys (§I.2); KSK ceremony discipline; ledger makes misuse visible	Compromise is detectable, not preventable — detection <i>*is*</i> the control
Customer key loss	Recovery options §III-C, chosen by red pen	Sovereignty maximalism = real loss; disclosed at signup
Chain reorg (if §III-E option A/B/C chosen)	Checkpoint depth requirements; ledger remains the primary record	Anchor is corroboration, never the sole record [DESIGN]
Descriptor harvesting / onion enumeration	v3 client auth on admin surfaces; portal addresses public by necessity	Public portals are public — admin surfaces are not
Downgrade: onion → clearnet	§II-B fail-closed: onion down = portal down	None by design
Insider at the registrar	Ledger daylight (§IV-A); separation of duties on ceremonies	Trust-but-verify, with the verify built in

Fail-closed table

Failure	Behavior
Ledger verification fails	Issuance halts; loud alert; no new registrations/transfers until resolved
Anchor checkpoint mismatch	Freeze on last verified checkpoint; investigate before advancing
Tor fails to bootstrap	Portal down; **no** clearnet fallback (§II-B)
Client-auth failure (SSH/admin)	Indistinguishable silence at the rendezvous — no "wrong key" oracle
TOFU key mismatch on returning customer	Connection refused loudly; support re-verification through a fresh ceremony

Failure	Behavior
DNSSEC signature expiry approaching	Automated rollover; expired signatures never served

§VII. The Four Wells — brief, genuine only

- **Hip-hop:** "They got me trapped / Can barely walk tha city streets / Without a cop harassing me, searching me / Then asking my identity" — 2Pac, "Trapped" (*2Pacalypse Now*, 1991) [VERIFIED FACT]. Identity demanded at gunpoint is the old world; identity *proved by key* — with nothing to seize — is this design's answer.
- **Law:** Black's 2d ed. (1910), s.v. "TRUST" — the registrar holds bare legal title; the equitable estate is the registrant's key (§IV-A) [VERIFIED FACT — public domain].
- **Religion:** "And ye shall know the truth, and the truth shall make you free." — John 8:32 (KJV) [VERIFIED FACT — public domain]. The ledger verifies, or it does not: TRVVTH alone judges.
- **Humanity:** "No one shall be subjected to arbitrary interference with his privacy, family, home or correspondence..." — Universal Declaration of Human Rights, Art. 12 [VERIFIED FACT]. Hence no central identity database (§III-A): what is never collected can never be seized, subpoenaed, or breached.

§VIII. Red-pen questions — his call alone

- **Chain choice:** Anchor to Bitcoin (Sidetree/ION or OpenTimestamps), an L2, our own chain, or no chain (ledger + DNSSEC only)? Survey in §III-E; no build begins without his ruling.
- **Recovery model:** Shamir-escrow (default, disclosed), social recovery, or sovereignty-maximalist no-recovery? §III-C.
- **Cleartext posture:** Onion-only portals, or optional cleartext mirror with Onion-Location? §II-B.
- **Root posture:** ICANN-root compatibility vs. independent/alternate root? Legal and interoperability consequences; counsel required either way. §V-C.
- **Billing identity:** Which payment processor/tokenization — and is DID + payment token sufficient identity for billing, or does he require more?
- **Relay posture:** Guard/middle relays at PoPs, or onion services only? §V-A.
- **PQ scope:** Hybrid PQ for all identity keys now, or root/recovery keys only? §I.2.

"What can occur — will." — Live, Love, and let Love, Live.

All Rights Reserved, Without Prejudice. CashApp \$axoneme